

SmartHire AI

Implementation Blueprint

1. PROJECT ARCHITECTURE

1.1 System Overview

SmartHire AI is a full-stack web application connecting job seekers with AI-powered resume analysis through React frontend and Node.js backend, leveraging Google's Gemini API for intelligent content evaluation and MySQL for data persistence on AWS.

1.2 Architecture Pattern

Three-tier architecture separating presentation (React), business logic (Node.js/Express), and data (MySQL) layers for independent scaling and maintenance.

1.3 Technology Stack Decisions

Component	Technology	Rationale
Frontend	React 18	Component reusability, virtual DOM, ecosystem maturity
Styling	Tailwind CSS	Utility-first, responsive, build-time optimization
Backend	Node.js + Express	JavaScript full-stack, non-blocking I/O, NPM ecosystem
File Processing	Multer + pdf-parse + mammoth	Specialized libraries for PDF and DOCX
AI Integration	Gemini 1.5 Pro	Natural language understanding, JSON output
Database	MySQL 8.0	Relational integrity, ACID compliance, RDS support
Cloud	AWS EC2 + RDS	Scalability, managed services, security
Process Manager	PM2	Production management, auto-restart, logging

2. PROJECT STRUCTURE

2.1 Directory Overview

Backend Structure:

```
text
backend/
├─ src/
│   ├─ config/          # database.js, gemini.js, multer.js, cors.js
│   ├─ models/          # Candidate, Resume, Analysis, Skill, Experience, Education
│   , Project, InterviewQuestion
│   ├─ controllers/     # analysisController, resumeController, recruiterController
│   ├─ services/        # fileParserService, aiService, analysisService, atsScoring
│   Service, reportService
│   ├─ routes/          # analysisRoutes, resumeRoutes, recruiterRoutes
│   ├─ middleware/      # fileValidation, errorHandler, ratelimiter, inputValidatio
n
│   ├─ utils/           # validators, helpers, logger, constants
│   └─ app.js           # Express initialization
├─ uploads/temp/        # Temporary file storage
├─ package.json
├─ .env
└─ ecosystem.config.js
```

Frontend Structure:

```
text
frontend/
├─ public/
│   └─ assets/          # Images, icons
├─ src/
│   ├─ components/
│   │   ├─ common/      # Button, Card, LoadingSpinner, ErrorBoundary, Notification
│   │   , ScoreDisplay, SkillTag
│   │   ├─ layout/      # Header, Footer, Layout
│   │   ├─ landing/     # HeroSection, FeatureGrid, CTASection
│   │   ├─ upload/      # FileUpload, ManualTextInput, UploadProgress
│   │   ├─ analysis/    # ATSScore, ResumeSummary, SkillsAnalysis, StrengthsWeaknes
ses, Suggestions, InterviewQuestions, ReadinessScore, DownloadReport
│   │   └─ recruiter/  # CandidateList, CandidateCard, CandidateDetails
```

```
|   ├── pages/           # LandingPage, UploadPage, AnalysisPage, RecruiterDashboard
|   ├── hooks/           # useFileUpload, useAnalysis, useNotification
|   ├── services/        # apiClient, analysisService, recruiterService
|   ├── utils/           # validators, formatters, constants
|   └── App.jsx
└── package.json
    └── tailwind.config.js
```

Database Structure:

```
text
database/
├── migrations/          # 001_candidates.sql through 008_interview_questions.sql
├── seeds/               # sample_data.sql
└── rollback/            # rollback.sql
```

2.2 Environment Configuration

Backend .env:

```
text
NODE_ENV=development
PORT=5000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=smarthire_ai
DB_PORT=3306
GEMINI_API_KEY=your_gemini_api_key
GEMINI_MODEL=gemini-1.5-pro
MAX_FILE_SIZE=10485760
RATE_LIMIT_WINDOW=900000
RATE_LIMIT_MAX=100
LOG_LEVEL=debug
CORS_ORIGIN=http://localhost:3000
```

Frontend .env:

```
text
REACT_APP_API_URL=http://localhost:5000/api/v1
REACT_APP_API_TIMEOUT=30000
```

3. DATABASE SCHEMA

3.1 Core Tables

Candidates Table:

Column	Type	Constraints	Purpose
candidate_id	INT	PRIMARY KEY AUTO_INCREMENT	Unique identifier
full_name	VARCHAR(100))	NOT NULL	Candidate name
email	VARCHAR(100))	NOT NULL UNIQUE	Contact email
phone	VARCHAR(20)		Contact number
location	VARCHAR(100))		City/State
linkedin_url	VARCHAR(255))		LinkedIn profile
portfolio_url	VARCHAR(255))		Portfolio website
professional_summary	TEXT		Brief summary
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Creation timestamp

updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE	Update timestamp
Resumes Table:			
Column	Type	Constraints	Purpose
resume_id	INT	PRIMARY KEY AUTO_INCREMENT	Unique identifier
candidate_id	INT	FOREIGN KEY	Links to candidate
file_name	VARCHAR(255)		Original filename
file_type	ENUM	NOT NULL	PDF, DOCX, or TEXT
file_size	INT		Size in bytes
extracted_text	LONGTEXT		Full extracted content
upload_source	ENUM	NOT NULL	FILE or MANUAL
upload_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Upload timestamp
Analyses Table:			
Column	Type	Constraints	Purpose
analysis_id	INT	PRIMARY KEY AUTO_INCREMENT	Unique identifier

resume_id	INT	FOREIGN KEY	Links to resume
ats_score	TINYINT	CHECK 0-100	ATS compatibility score
ats_breakdown	TEXT		Score breakdown details
resume_summary	TEXT		Generated summary
strengths	TEXT		List of strengths
weaknesses	TEXT		List of weaknesses
missing_skills	TEXT		Missing skills
suggestions	TEXT		Improvement suggestions
interview_readiness_score	TINYINT	CHECK 0-100	Interview readiness
readiness_explanation	TEXT		Score explanation
ai_response_raw	LONGTEXT		Raw AI response

created_at	TIMESTAM P	DEFAULT CURRENT_TIMESTAMP P	Analysis timestamp
------------	---------------	-----------------------------------	-----------------------

3.2 Supporting Tables

Skills Table:

- skill_id (PK), candidate_id (FK), skill_name, skill_category (Technical/Soft/Other), proficiency_level (Beginner/Intermediate/Advanced/Expert), is_missing (BOOLEAN), priority (1-5)

Experiences Table:

- experience_id (PK), candidate_id (FK), job_title, company_name, location, start_date, end_date, current_role (BOOLEAN), description, achievements

Educations Table:

- education_id (PK), candidate_id (FK), degree, field_of_study, institution, location, start_date, end_date, gpa (0-4.0)

Projects Table:

- project_id (PK), candidate_id (FK), project_name, description, technologies, outcomes, project_url

InterviewQuestions Table:

- question_id (PK), analysis_id (FK), question_text, question_type (Technical/Behavioral/Situational/General), difficulty (Easy/Medium/Hard/Expert), suggested_answer, category

3.3 Relationships Summary

- Candidate ↔ Resume: One-to-One
- Resume ↔ Analysis: One-to-One
- Candidate ↔ Skills: One-to-Many
- Candidate ↔ Experiences: One-to-Many
- Candidate ↔ Educations: One-to-Many
- Candidate ↔ Projects: One-to-Many
- Analysis ↔ InterviewQuestions: One-to-Many

3.4 Index Strategy

Performance Indexes: candidates.email, candidates.full_name, resumes.candidate_id, analyses.resume_id, skills.candidate_id, [skills.is_missing](#), experiences.candidate_id, interview_questions.analysis_id

Composite Indexes: experiences(candidate_id, start_date), analyses(resume_id, created_at)

4. API SPECIFICATION

4.1 API Architecture

- **Base URL:** `http://localhost:5000/api/v1` (Dev)
`/ https://api.smarthire.com/v1` (Prod)
- **Authentication:** None (MVP)
- **Rate Limiting:** 100 requests per 15 minutes per IP
- **Response Format:** JSON with consistent structure

4.2 Core Endpoints

POST `/api/v1/analysis/upload-file`

- **Purpose:** Upload and analyze resume file
- **Content-Type:** `multipart/form-data`
- **Parameters:** `resume` (File, required), `name` (String, required), `email` (String, required)
- **Success (201):** Returns complete analysis with ATS score, summary, strengths, weaknesses, missing skills, suggestions, interview questions, readiness score
- **Error Codes:** `FILE_TYPE_INVALID` (400), `FILE_TOO_LARGE` (400), `INVALID_INPUT` (400), `AI_SERVICE_ERROR` (500)

POST `/api/v1/analysis/upload-text`

- **Purpose:** Upload and analyze manual resume text
- **Content-Type:** `application/json`
- **Body:** `resume_text` (100-50000 chars), `candidate_name`, `candidate_email`
- **Success (201):** Same as file upload
- **Error Codes:** `TEXT_TOO_SHORT` (400), `TEXT_TOO_LONG` (400), `INVALID_EMAIL` (400)

GET /api/v1/analysis/:id

- **Purpose:** Retrieve analysis by ID
- **Success (200):** Returns analysis with candidate details
- **Error:** ANALYSIS_NOT_FOUND (404)

GET /api/v1/analysis/:id/report

- **Purpose:** Download analysis report as HTML
- **Success (200):** HTML document with complete analysis
- **Error:** ANALYSIS_NOT_FOUND (404)

GET /api/v1/recruiter/candidates

- **Purpose:** Get paginated candidate list with analysis summary
- **Query Parameters:** page, limit, search, sort_by, sort_order
- **Success (200):** Returns candidates with ATS and interview scores

GET /api/v1/recruiter/candidates/:id

- **Purpose:** Get complete candidate profile
- **Success (200):** Returns candidate with analysis, skills, experiences, educations, projects

4.3 Error Codes Reference

Code	Status	Description	Action
FILE_TYPE_INVALID	400	Unsupported file type	Upload PDF or DOCX
FILE_TOO_LARGE	400	File exceeds 10MB	Compress or use manual input
TEXT_TOO_SHORT	400	Text under 100 chars	Provide more content
TEXT_TOO_LONG	400	Text over 50000 chars	Shorten text
INVALID_EMAIL	400	Invalid email format	Provide valid email
ANALYSIS_NOT_FOUND	404	Analysis ID not found	Verify ID
RATE_LIMIT_EXCEEDED	429	Too many requests	Wait 15 minutes
AI_SERVICE_ERROR	500	Gemini API error	Retry with backoff

DATABASE_ERROR	500	Database failure	Contact support
SERVICE_UNAVAILABLE	503	Service temporarily down	Retry after 5 minutes

5. COMPONENT ARCHITECTURE

5.1 Frontend Components

Common Components:

Component	Props	Functionality
Button	variant, size, loading, disabled, onClick	Primary/Secondary/Outline with loading state
Card	title, subtitle, children	Container with header and content
LoadingSpinner	size, color, text	Centered spinner with optional message
ErrorBoundary	fallback, children	Catches React errors, shows fallback UI
Notification	type, message, duration	Toast notifications (success/error/warning)
ScoreDisplay	score, label, size, breakdown	Circular progress (0-40 Red, 41-70 Orange, Green)
SkillTag	name, category, level	Colored tag with proficiency indicator

Page Components:

Page	Sections	Functionality
LandingPage	Hero, Features, How It Works, CTA	Marketing content and call to action
UploadPage	File Upload, Manual Input, Submit	Two upload options with validation

AnalysisPage	Loading, Results Overview, All Sections	Complete analysis display with s
RecruiterDashboard	Candidate List, Search, Sort, Details	Browse candidates and view det

5.2 Backend Components

Controllers:

Controller	Methods	Purpose
AnalysisController	uploadFile, uploadText, getAnalysis, downloadReport	Handle analysis op
RecruiterController	getCandidates, getCandidateDetails	Handle recruiter op

Services:

Service	Methods	Dependencies
FileParserService	parsePDF, parseDOCX, extractStructuredData	pdf-parse, ma
AIService	analyzeResume, generateQuestions, handleRetry	Gemini API
AnalysisService	processAnalysis, calculateATSScore, generateSummary	AIService
ATSScoringService	calculateScore, generateBreakdown	Constants
ReportService	generateHTMLReport, formatForPrint	AnalysisService

Middleware:

Middleware	Purpose	Configuration
fileValidation	Validate file type and size	PDF/DOCX, max 10MB
errorHandler	Centralized error handling	Logs errors, standardized responses

rateLimiter	API rate limiting	100 requests per 15 minutes
inputValidation	Validate request inputs	Email, name, text length

6. IMPLEMENTATION PLAN

Day-by-Day Breakdown

Day 1: Project Setup & Database

- Create complete folder structure
- Initialize backend with Express, frontend with React
- Set up MySQL database and run all migrations
- Configure environment variables
- **Deliverable:** Working project structure with database

Day 2: Models & Services

- Implement all database models (Candidate, Resume, Analysis, Skill, Experience, Education, Project, InterviewQuestion)
- Create file parser service for PDF and DOCX
- Implement AI service with Gemini API integration
- **Deliverable:** All models and core services functional

Day 3: Controllers & API

- Implement all controllers (analysisController, recruiterController)
- Create complete REST API endpoints with validation
- Add middleware for file validation, rate limiting, error handling
- **Deliverable:** All API endpoints tested and working

Day 4: Frontend Components

- Set up React Router with all pages
- Create layout components (Header, Footer, Layout)
- Build landing page with hero, features, CTA
- **Deliverable:** All pages rendered with routing

Day 5: Upload & Analysis

- Implement file upload with drag-and-drop
- Create manual text input with validation
- Build all analysis components (ATS score, summary, skills, strengths/weaknesses, suggestions, questions, readiness)
- **Deliverable:** Complete upload and analysis flow

Day 6: Recruiter Dashboard

- Implement candidate list with search and sort
- Create candidate card and details view
- Build report generation and download
- **Deliverable:** Full recruiter dashboard functional

Day 7: Testing & QA

- Unit testing for frontend and backend
- Integration testing for all APIs
- End-to-end testing of complete workflows
- Edge cases validation
- **Deliverable:** All tests passing, bugs fixed

Day 8: Deployment Preparation

- Configure AWS EC2 instance with security groups
- Set up AWS RDS MySQL database
- Prepare production environment variables
- Create PM2 and Nginx configuration

- **Deliverable:** Production environment ready

Day 9: Deployment

- Deploy backend to EC2 with PM2
- Deploy frontend build to Nginx
- Run database migrations on RDS
- Configure monitoring and logging
- **Deliverable:** Application live on AWS

Day 10: UAT & Finalization

- Conduct user acceptance testing
- Validate all features in production
- Complete documentation
- Finalize project deliverables
- **Deliverable:** Project complete and ready for submission

7. AI INTEGRATION SPECIFICATIONS

7.1 Gemini API Configuration

- **Model:** gemini-1.5-pro
- **Timeout:** 30 seconds
- **Retry Policy:** 3 retries with exponential backoff

7.2 Prompt Structure

text

You are an expert resume evaluator. Analyze the following resume and provide comprehensive evaluation with actionable insights.

RESUME CONTENT: {extracted_text}

- Provide:
1. ATS compatibility score (0-100) with breakdown
 2. Professional summary
 3. Strengths (list with brief explanation)
 4. Weaknesses (list with brief explanation)
 5. Missing skills (with priority 1-5)
 6. Improvement suggestions (with priority 1-5)
 7. Interview readiness score (0-100) with explanation
 8. 10 interview questions (technical, behavioral, situational)

Format as JSON with specific schema.

7.3 ATS Scoring Factors

Factor	Weight	Description
Keyword Density	25%	Comparison against industry standards
Formatting & Structure	20%	Section headers, consistency
Content Completeness	20%	Presence of all required sections

Factor	Weight	Description
Grammar & Spelling	10%	Accuracy and professionalism
Action Verb Usage	10%	Strong, varied action verbs
Quantification	10%	Metrics and achievements
Contact Information	5%	Complete and professional

Score Interpretation:

- 90-100: Excellent - Well-optimized for ATS
- 75-89: Good - Room for improvement
- 60-74: Fair - Some issues affecting performance
- 0-59: Needs Improvement - Significant optimization needed

8. DEPLOYMENT GUIDE

8.1 AWS EC2 Setup

Instance Configuration:

- AMI: Ubuntu 22.04 LTS
- Instance Type: t2.micro (Free tier)
- Storage: 20GB GP2 SSD

Security Groups:

Port	Protocol	Source	Purpose
22	SSH	Developer IP	SSH access
80	HTTP	0.0.0.0/0	Web access
443	HTTPS	0.0.0.0/0	Secure web access
5000	TCP	0.0.0.0/0	Application access

Setup Commands:

```
bash
# Update system
sudo apt update && sudo apt upgrade -y

# Install Node.js 18
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs

# Install PM2
sudo npm install -g pm2

# Install Nginx
sudo apt install -y nginx
```

8.2 AWS RDS Setup

Database Configuration:

- Engine: MySQL 8.0
- Instance Class: db.t2.micro
- Storage: 20GB
- Backup Retention: 7 days
- Public Accessibility: No

Connection Details:

text

DB_HOST: smarthire-db.xxxxxxxxxxxx.rds.amazonaws.com

DB_USER: admin

DB_PASSWORD: [YourSecurePassword]

DB_NAME: smarthire_ai

DB_PORT: 3306

8.3 Production Environment Variables

text

NODE_ENV=production

PORT=5000

DB_HOST=smarthire-db.xxxxxxxxxxxx.rds.amazonaws.com

DB_USER=admin

DB_PASSWORD=YourSecurePassword!

DB_NAME=smarthire_ai

DB_PORT=3306

GEMINI_API_KEY=your-gemini-api-key

GEMINI_MODEL=gemini-1.5-pro

CORS_ORIGIN=http://your-domain.com

MAX_FILE_SIZE=10485760

LOG_LEVEL=info

8.4 PM2 Configuration

javascript

```
module.exports = {
```

```
  apps: [{
```

```

    name: 'smarthire-api',
    script: 'src/app.js',
    instances: 1,
    exec_mode: 'fork',
    env: {
      NODE_ENV: 'production',
      PORT: 5000
    },
    error_file: '/var/log/smarthire/err.log',
    out_file: '/var/log/smarthire/out.log',
    max_memory_restart: '1G',
    merge_logs: true
  }]
};

```

8.5 Nginx Configuration

```

nginx
server {
    listen 80;
    server_name your-domain.com;

    location /api {
        proxy_pass http://localhost:5000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        root /var/www/html;
        try_files $uri /index.html;
    }
}

```

8.6 Deployment Steps

1. Clone repository on EC2
2. Install backend dependencies
3. Run database migrations on RDS
4. Build and deploy frontend to Nginx
5. Start application with PM2
6. Configure monitoring and logging

9. TESTING STRATEGY

9.1 Unit Testing

- **Frontend:** Component rendering, user interactions, API calls, utilities
- **Backend:** Model CRUD operations, service methods, validation functions

9.2 Integration Testing

- API endpoint testing with Supertest
- Database integration tests
- File upload tests
- AI service mock tests

9.3 System Testing

End-to-End Scenarios:

1. Complete candidate workflow (upload → analysis → report)
2. Manual text input workflow
3. Recruiter dashboard access and viewing
4. File upload validation (invalid files, large files)
5. Error handling and recovery

Performance Testing:

- Load: 50 concurrent users
- Stress: 200 concurrent users
- Response time verification

9.4 Edge Cases

Edge Case	Expected Behavior
Empty PDF	Error: "Empty file"
Corrupted PDF	Error: "File corrupted"
Scanned PDF	Error: "No text extracted"
Complex DOCX	Text extracted successfully
HTML injection	Sanitized before processing
Special characters	Displayed correctly
Very long experience	Handled without truncation
Multiple pages	Processed within timeout
Missing contact info	Partial analysis with warning
AI service timeout	Retry with error message
Database connection lost	Graceful error response
Rapid uploads	Rate limiting applies

10. SECURITY IMPLEMENTATION

10.1 Security Layers

Input Validation:

- File type validation (PDF/DOCX)
- File size validation (max 10MB)
- Text length validation (100-50000 chars)
- Email format validation
- SQL injection prevention via parameterized queries

File Security:

- MIME type verification
- File extension validation
- Temporary storage with auto-cleanup
- Random filename generation

Application Security:

- Helmet.js security headers
- CORS configuration
- Rate limiting (100 requests/15 minutes)
- Express-validator for request validation
- XSS prevention through input sanitization

Infrastructure Security:

- VPC network isolation
- Security group restrictions
- RDS in private subnet
- SSL encryption for data in transit
- Environment variables for secrets

10.2 Security Headers

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- X-XSS-Protection: 1; mode=block
- Content-Security-Policy: default-src 'self'
- Strict-Transport-Security: max-age=31536000

11. MONITORING & LOGGING

11.1 Log Levels

- **ERROR:** Critical failures, database errors, AI failures
- **WARN:** Potential issues, validation warnings
- **INFO:** User actions, API calls, successful operations
- **DEBUG:** Detailed debugging (development only)

11.2 Key Metrics

Application Metrics:

- API response time (target: < 2 seconds)
- Analysis processing time (target: < 30 seconds)
- File upload success rate (target: > 95%)
- Error rate (target: < 5%)

Infrastructure Metrics:

- CPU utilization (alert at 80%)
- Memory usage (alert at 85%)
- Database connection pool usage
- Disk space utilization

AI Service Metrics:

- API call success rate
- Response time
- Rate limit usage
- Token usage

11.3 CloudWatch Alarms

- CPU > 80%
- Memory > 85%
- Error Rate > 5%
- Response Time > 3 seconds

12. PROJECT COMPLETION CHECKLIST

Development

- ☐

All backend models implemented

- ☐

All services implemented

- ☐

All controllers complete

- ☐

All API routes configured

- ☐

All middleware in place

- ☐

All React pages created

- ☐

All React components built

- ☐

Routing configured

- ☐

API integration complete

- ☐

Responsive design implemented

Database

- ☐

All tables created

- ☐

Relationships established

- ☐

Indexes created

- ☐

Foreign keys configured

- ☐

Test data inserted

AI Integration

- ☐

Gemini API configured

- ☐

Prompts engineered and optimized

- ☐

Response parsing complete

- ☐

Error handling in place

- ☐

Retry logic implemented

Testing

- ☐

Unit tests passing

- ☐

Integration tests passing

- ☐

Manual testing complete

- ☐

Edge cases tested

- ☐

Bug fixes applied

Deployment

- ☐

AWS EC2 configured

- ☐

AWS RDS configured

- ☐

Environment variables set

- ☐

Production build created

- ☐

Application running

- ☐

Monitoring configured

Documentation

- ☐

README complete

- ☐

Architecture documentation

- ☐

API specification

- ☐

Deployment guide

- ☐

Testing guide

Final Deliverables

- ☐

Complete source code

- ☐

Database schema

- ☐

Postman collection

- ☐

Screenshots of application

- ☐

Project report/documentation

13. SUCCESS METRICS

Technical Performance

Metric	Target	Measurement
API Response Time	< 2 seconds	CloudWatch
Page Load Time	< 3 seconds	Lighthouse
System Uptime	99.5%	AWS CloudWatch
Error Rate	< 5%	Application logs
AI Processing	< 30 seconds	Custom metrics

Functional Metrics

Feature	Success Criteria	Verification
File Upload	> 95% success rate	Upload tracking
Resume Analysis	100% completion	Database records
Report Generation	100% of analyses	Feature tracking
Recruiter Access	100% of candidates	Database query

Quality Metrics

Aspect	Target	Measurement
Code Coverage	> 80%	Jest coverage

Aspect	Target	Measurement
Accessibility	WCAG 2.1 AA	Lighthouse
Performance	> 90	Lighthouse
Best Practices	> 90	Lighthouse

14. TROUBLESHOOTING GUIDE

Common Issues

Database Connection:

- Symptom: "ECONNREFUSED" or "Access denied"
- Solution: Verify credentials, check MySQL is running, ensure correct host

File Upload:

- Symptom: "File type not supported" or "File too large"
- Solution: Check file extension, verify file size, ensure proper MIME types

AI Service:

- Symptom: "AI_SERVICE_ERROR" or timeout
- Solution: Verify API key, check internet, implement retry logic

Frontend Build:

- Symptom: Build fails or dependencies missing
- Solution: Clear npm cache, delete node_modules, reinstall dependencies

Deployment:

- Symptom: Application not accessible
- Solution: Check security groups, verify Nginx config, check PM2 logs

Debugging Commands

```
bash
```

```
# Check PM2 Logs
```

```
pm2 logs smarthire-api
```

```
# Check Nginx Logs
```

```
sudo tail -f /var/log/nginx/error.log
```

```
# Check database connection
```

```
mysql -h endpoint -u admin -p -e "SELECT 1"
```

```
# Test API endpoint
```

```
curl http://localhost:5000/api/v1/health
```

```
# Check Node.js version
```

```
node --version
```

15. FUTURE ENHANCEMENTS

Version 1.1

- Authentication (JWT-based)
- User registration and login
- Password management
- Role-based access control
- Multiple resume management
- Resume version history
- Email notifications

Version 2.0

- Job posting and management
- Candidate matching algorithm
- Automated screening
- Interview scheduling
- Advanced analytics dashboard
- LinkedIn integration
- Mobile application

Version 3.0

- Team collaboration
- Workflow automation
- Custom reporting
- AI model fine-tuning
- Multi-language support
- White-label solutions
- Enterprise features

Document Version: 1.0

Last Updated: July 2026

Prepared By: Tarunteja Patro

Status: Final - Ready for Implementation